Train as You Evict: Eviction-Aware Fine-Tuning for Compressed KV Cache Inference

Anonymous TACL submission

Abstract

KV cache eviction is a practical way to control the memory and computational cost of long-context inference in transformer language models. However, eviction methods are typically applied post hoc at inference time, whereas the model itself is trained under standard full-cache retention. Because eviction alters the model's effective conditioning history during decoding, it introduces a train-test mismatch. We study this mismatch and evaluate fine-tuning under eviction as a simple adaptation strategy. On a long-context question-answering benchmark, fine-tuning with eviction active substantially improves performance under evicted-cache inference compared with applying the same eviction method only after training. Further analyses suggest that these gains arise primarily from adaptation specialised to the evicted-cache regime. Together, our findings suggest that shifts in the model's effective conditioning context are an important consideration in KV cache eviction and that eviction-aware training is a promising approach for improving memory-compressed inference.

1 Introduction

Large transformer-based models have become the backbone of a wide range of modern AI systems (Vaswani et al., 2017). A key strength of these models lies in their ability to incorporate information and adapt their behaviour through the conditioning context, which can strongly influence whether a model succeeds or fails on a given task (Brown et al., 2020; Liu et al., 2024a). Because context is limited, curating the information made available to the model has become central to improving performance in these systems. Although recent work has pushed toward ever larger context windows, longer contexts come at a significant cost in memory and computation and can substantially slow inference (Dao et al., 2022; Zaheer et al., 2020). As a result, token eviction mechanisms that selectively remove entries from the model's KV cache have emerged as a practical approach to controlling context growth while attempting to preserve the information most relevant for future predictions (Xiao et al., 2024; Zhang et al., 2023; Liu et al., 2023; Li et al., 2024; Chen et al., 2024).

These methods are typically applied post hoc to trained transformer models as an inference-time optimisation. Although this approach can provide practical benefits, it also introduces a distribution shift in the model's effective conditioning context. By systematically altering the model's conditioning history, KV cache pruning methods place the model in an effective conditioning regime for which it was not explicitly optimised, suggesting a train-test mismatch that may benefit from adaptation under the target regime (Moreno-Torres et al., 2012; Gururangan et al., 2020). In this work, we study the train-test mismatch induced by token eviction strategies and the extent to which it limits transformer-based language models from fully realising the benefits from otherwise effective context pruning. We propose fine-tuning under eviction as a way to mitigate this mismatch and investigate its effectiveness as a practical adaptation strategy.

On a long-context question answering benchmark, we find that applying eviction only at inference time substantially underperforms fine-tuning with the eviction method active during training. We further analyse the resulting models to examine what kind of adaptation eviction-aware fine-tuning induces and find evidence that its gains are associated with changes in internal processing that are specialised for the evicted-cache regime.

Taken together, these results point to a distribution shift in the model's effective conditioning as an important and relatively underexplored consequence of KV cache eviction and suggest that adapting the model under the compressed-cache regime can be a simple and effective way to improve performance under memory-compressed inference.

2 Related Work

An effective approach to controlling KV cache growth is token eviction, which retains only a subset of cached key-value pairs according to an estimate of their importance. Existing methods differ both in the criteria used to determine which entries to keep and in when those retention decisions are made. Some rely on decoding-time heuristics, typically combining recency with attention-derived measures of token importance (Xiao et al., 2024; Zhang et al., 2023; Liu et al., 2023), while others make retention decisions during or shortly after prefill, using saliency or attention patterns observed before generation to select a fixed subset of entries for subsequent decoding (Li et al., 2024; Chen et al., 2024). More recently, learned eviction mechanisms have been proposed as alternatives to hand-designed heuristics (Cetin et al., 2025). Complementary work has explored other forms of KV cache compression, including quantization (Liu et al., 2024b), low-rank compression (Chang et al., 2025), and merging (Wang et al., 2024). Despite these methodological differences, existing compression methods are typically applied post hoc at inference time to a frozen base model and therefore do not directly address the fact that eviction places the model in a conditioning regime for which it was not explicitly optimised.

A large body of work has studied distribution shift and methods for adapting models to target conditions that differ from those seen during optimisation (Pan and Yang, 2009; Moreno-Torres et al., 2012). Much of this literature focuses on shifts in task domain or, more generally, in the observed input distribution. Our setting is related but distinct: the mismatch can arise even when the input token sequence itself is unchanged. Instead, the shift occurs in the model's effective conditioning context, because eviction mechanisms alter the retained memory state available for prediction. In autoregressive language modelling, exposure bias is a separate and well-studied source of train-test mismatch (Bengio et al., 2015), but it is not the primary discrepancy we study here. A widely studied approach to handling distribution shift is to fine-tune models under the target regime (Gururangan et al., 2020). For large foundation models, such adaptation can often be carried out efficiently using parameter-efficient methods (Houlsby et al., 2019; Hu et al., 2022). Our work investigates this adaptation strategy specifically for the conditioning shift induced by KV cache eviction.

Prior work has shown that KV cache eviction can substantially degrade performance and has largely explained this in terms of imperfect eviction decisions: token-eviction policies rely on imperfect proxy measures of token importance, and discarding useful entries causes irreversible information loss (Zhang et al., 2023; Liu et al., 2023; Zhang et al., 2025). Our focus, instead, is on the additional degradation that can arise when applying an eviction method post hoc because the model is evaluated under an effective conditioning regime for which it was not explicitly trained, a source of mismatch that is distinct from the quality of the eviction policy itself. Closest to our setting, G-KV argues that pretrained models may be suboptimal under compressed KV cache inference. Thus, it introduces post-training methods to adapt them to this regime, using reinforcement learning for tasks with verifiable rewards and distillation to train the compressed-cache model to match the full-cache model otherwise (Liao et al., 2025). In contrast, our work formulates this problem more explicitly as a source of distribution shift in the base model's conditioning context, and investigates whether standard log-likelihood training under eviction is sufficient to mitigate it, while also providing a complementary analysis of this source of train-test mismatch.

3 Method

3.1 Preliminaries / Background

In decoder-only transformers with causal self-attention, network activations for a given timestep are not affected by future input tokens. This removes the need to recompute past activations when generating new outputs. In particular, exact decoding for step $t+1$ only requires the new input token x_{t+1} together with the keys and values of past tokens. This motivates KV-caching. Formally, let $K_t^{(\ell,h)}$ and $V_t^{(\ell,h)}$ denote the cached

keys and values for layer ℓ and head h after processing the first t tokens. At decoding step $t+1$, the model receives the current hidden state for the new token, computes its query $q_{t+1}^{(\ell,h)}$, key $k_{t+1}^{(\ell,h)}$, and value $v_{t+1}^{(\ell,h)}$, and updates the cache:

$$\begin{aligned} K_{t+1}^{(\ell,h)} &= \left[K_t^{(\ell,h)};\, k_{t+1}^{(\ell,h)}\right] \\ V_{t+1}^{(\ell,h)} &= \left[V_t^{(\ell,h)};\, v_{t+1}^{(\ell,h)}\right] \end{aligned} \tag{1}$$

The attention for the new token is then computed against the full cached history:

$$\begin{aligned} a_{t+1}^{(\ell,h)} &= \mathrm{softmax}\left(\frac{q_{t+1}^{(\ell,h)}(K_{t+1}^{(\ell,h)})^\top}{\sqrt{d_k}}\right), \\ o_{t+1}^{(\ell,h)} &= a_{t+1}^{(\ell,h)} V_{t+1}^{(\ell,h)}. \end{aligned} \tag{2}$$

Where d_k is the key dimension, and $o_{t+1}^{(\ell,h)}$ is the attention output. This yields the exact same result as recomputing the full prefix at each step, while avoiding repeated work.

The KV cache acts as the transformer's memory, growing with the amount of processed context. As its size increases, it becomes a bottleneck for memory usage and inference speed. Memory eviction mechanisms address this by selectively removing entries from the cache during decoding to limit resource usage while retaining the information most relevant for future predictions.

To formalise this, we model an eviction strategy as an operator E that dynamically selects which tokens to retain. At decoding step $t+1$, for each layer ℓ and head h, the strategy determines a subset of retained indices $I_{t+1}^{(\ell,h)} = E^{(\ell,h)}(\tilde{K}_t, \tilde{V}_t, \mathcal{X}_{t+1})$, where $\tilde{K}_t$ and $\tilde{V}_t$ denote the global cache states across all layers and heads, and $\mathcal{X}_{t+1}$ represents any auxiliary inputs used to inform the policy. The resulting evicted cache is:

$$\begin{aligned} \tilde{K}_{t+1}^{(\ell,h)} &= \left[\tilde{K}_t^{(\ell,h)};\, k_{t+1}^{(\ell,h)}\right][I_{t+1}^{(\ell,h)}] \\ \tilde{V}_{t+1}^{(\ell,h)} &= \left[\tilde{V}_t^{(\ell,h)};\, v_{t+1}^{(\ell,h)}\right][I_{t+1}^{(\ell,h)}] \end{aligned} \tag{3}$$

Thus, an eviction strategy modifies autoregressive generation by continually pruning the model's memory.

3.2 Eviction as a Source of Distribution Shift

Memory eviction modifies autoregressive decoding by altering the conditioning history available to the model at each step. Under standard full-cache decoding, the model conditions at step t on the complete cache (K_t, V_t) associated with the observed prefix. Under an eviction strategy, it instead conditions on the compressed cache $(\tilde{K}_t, \tilde{V}_t)$. This induces a distribution shift in the effective conditioning histories encountered during decoding, independently of any shift in the distribution of raw input tokens. This in turn impacts the model's output distribution:

$$\begin{aligned} p_\theta(x_{t+1} \mid x_{1:t}) &= p_\theta(x_{t+1} \mid K_t, V_t) \\ &\neq p_\theta(x_{t+1} \mid \tilde{K}_t, \tilde{V}_t) \\ &= p_\theta^E(x_{t+1} \mid x_{1:t}) \end{aligned} \tag{4}$$

This mismatch can be problematic because eviction is typically applied post hoc at inference time, while model parameters θ are optimised under standard full-cache retention. As a result, the model is evaluated under a conditioning regime for which its parameters were not explicitly trained. Furthermore, because autoregressive decoding feeds generated tokens back as inputs, local perturbations in the next-token distribution can compound over time. The mismatch induced by eviction can therefore exacerbate the effects of standard exposure bias.

3.3 Adaptation to Eviction-Induced Shift

The train-test mismatch induced by memory eviction can in principle be addressed by training the model directly under the modified distribution. For large language models, however, training from scratch is often computationally infeasible. Starting from a pre-trained model, fine-tuning provides a standard and computationally practical way to adapt to distribution shift. Concretely, rather than applying eviction solely post hoc at inference time, we fine-tune the base LLM with the eviction strategy active during training. Given a training corpus $\mathcal{D}$ of token sequences $x_{1:T}$, we optimise the standard autoregressive negative log-likelihood under the modified conditioning distribution induced by eviction:

$$\mathcal{L}_E(\theta) = -\mathbb{E}_{x_{1:T}\sim\mathcal{D}}\left[\sum_{t=1}^{T-1} \log p_\theta^E(x_{t+1} \mid x_{1:t})\right] \tag{5}$$

While this objective is straightforward when the eviction strategy is model-agnostic, many modern methods use fixed strategies whose eviction decisions depend on the model's internal activations. In that setting, fine-tuning the LLM changes the representations used by the eviction rule, so the effective conditioning distribution induced by eviction can itself drift during optimisation. In this work, we do not introduce additional machinery to account for this dependence. Instead, for fixed model-dependent eviction strategies, we directly optimise Eq. (1) and leave it to experiments to assess whether this is sufficient in practice.

4 Experiments

Our experimental study is centred around three main questions. First, does fine-tuning with eviction active improve performance under evicted-cache inference (Sec. 5.1)? Second, what kind of adaptation does eviction-aware fine-tuning induce, and how does the resulting model differ from one fine-tuned under standard full-cache conditioning (Sec. 5.2)? Third, when the eviction policy is frozen but depends on model activations, to what extent does fine-tuning change the resulting eviction behaviour, and is this change large enough to threaten training stability (Sec. 5.3)?

4.1 Experimental Setup

We conduct our experiments using a pre-trained Llama 3.2 1B model as the base LLM. As our memory eviction mechanism, we use a Neural Attention Memory Model (NAMM) (Cetin et al., 2025). NAMM is a neural network trained with CMA-ES to maximise performance on a language modelling objective. Evolutionary optimisation is a natural fit for this setting, as it avoids the difficulties posed by the non-differentiable memory-management decisions involved in selecting which tokens to retain or discard, as well as by non-differentiable evaluation objectives such as the F1 score used in our experiments.

At each layer and attention head, the eviction module computes features from the corresponding attention activations and then predicts which past tokens should be removed from that head's KV cache. The NAMM policy is applied independently to each head in each layer after every 256 processed tokens. Note that our implementation of the NAMM differs from the one proposed by Cetin et al. (2025); we describe the differences in Appendix C and, in particular, use a cache-size-based eviction rule rather than the threshold-based rule of the original paper (Appendix C.1). All adaptations of the pre-trained LLM are performed using LoRA for parameter-efficient fine-tuning. Additional hyperparameter and implementation details are provided in Appendix B.

For clarity, we distinguish between the model variant and the inference regime used in each experiment. We consider three model variants: **Base**, the original pre-trained Llama 3.2-1B model; **FTS** (*fine-tuned standard*), obtained by fine-tuning the base model with standard full-cache decoding during training; and **FTE** (*fine-tuned with eviction*), obtained by fine-tuning with the NAMM eviction policy active during training. We also consider two inference regimes: **FC** (*full cache*), in which decoding uses the standard uncompressed KV cache, and **EC** (*evicted cache*), in which the NAMM policy is applied at inference time. We denote each evaluated configuration by combining these two components. For example, FTS-EC refers to the model fine-tuned under standard full cache decoding but evaluated with eviction.

4.2 Datasets and Evaluation Protocol

Our benchmark is constructed from question-answering examples drawn from Qasper (Dasigi et al., 2021), 2WikiMultihopQA (Ho et al., 2020), and HotpotQA (Yang et al., 2018) in LongBench (Bai et al., 2024), together with additional Qasper and 2WikiMultihopQA sequences from LongBench-E. To focus on sufficiently long contexts while keeping training computationally manageable, we retain only examples whose tokenised length lies between 4096 and 6500 tokens. After filtering, this yields 440 sequences in total, with each source contributing between 73 and 111 sequences. We split this dataset into training, validation, and test sets using a stratified 70/15/15 partition over the five data sources. During training, we assign each example a sampling weight inversely proportional to the size of its source dataset to compensate for the remaining source imbalance.

In addition to this main benchmark, we construct a separate long-context evaluation set consisting of 154 examples from the same sources with lengths between 6500 and 8192 tokens. We use this set only for out-of-distribution evaluation, allowing us to assess generalisation beyond the training length range.

We use F1 as the evaluation metric throughout. Model checkpoints are selected based on validation F1, and all validation and test evaluations are performed with temperature zero.

5 Results & Discusion

5.1 Performance of Eviction-Aware Training

Our main results on the QA benchmark introduced in Section 4.2 are shown in Figure 1. Standard task-specific fine-tuning substantially improves performance over the pre-trained model under full-cache decoding, as reflected in the gap between Base-FC and FTS-FC. When the NAMM eviction policy is applied only at inference time, however, performance drops sharply for both the base and fine-tuned models. In particular, the drop from FTS-FC to FTS-EC is larger than the gain from Base-FC to FTS-FC, highlighting the scale of the degradation introduced by post-hoc eviction. By contrast, fine-tuning with eviction active during training substantially mitigates the performance loss caused by inference-time eviction: FTE-EC markedly outperforms FTS-EC and recovers much of the performance achieved by FTS-FC. These results suggest that adapting the model under the evicted-cache regime is an effective way to reduce the performance degradation introduced by post-hoc eviction. Notably, this recovery is obtained under a compression setting in which each head retains at most 1024 cached tokens, even though the evaluation prompts contain 4096-6500 tokens.

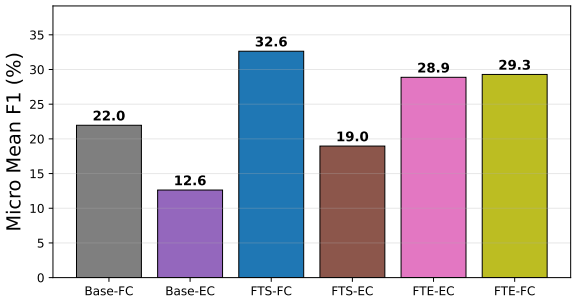

Figure 1: Micro-mean F1 on the QA benchmark test set across all evaluated models and inference configurations.

5.2 Performance Attribution Analysis

We next examine what kind of adaptation is induced by fine-tuning with eviction active. Figure 1 shows that FTE-FC performs very similarly to FTE-EC, despite having access to the full cache, while both remain below FTS-FC and substantially outperform the pre-trained baseline. This pattern suggests that eviction-aware fine-tuning improves the model, but not in a way that yields uniformly better performance across inference settings. Instead, it induces an adaptation that helps the model operate effectively under the evicted-cache regime while sacrificing part of the full-cache gains achieved by standard fine-tuning.

Next Token Logit Distribution Shift To investigate our claim of specialisation towards an eviction regime, we compare the distribution shift of the predicted next token logit between pairs FTS-FC/FTS-EC and FTE-FC/FTE-EC to try and isolate the influence of the eviction policy during training. Figure 2 reports the mean JS divergence of the logit distributions over generation across all validation and test prompts. We observe comparable distribution shifts in both cases, both significant but smaller than that eviction on the base model (Base-FC/Base-EC). This suggests that eviction-aware fine-tuning (FTE) learns mostly a specialised adaptation, though the reduction in distribution shift compared to Base-FC/Base-EC suggestions shows some level of eviction robustness.

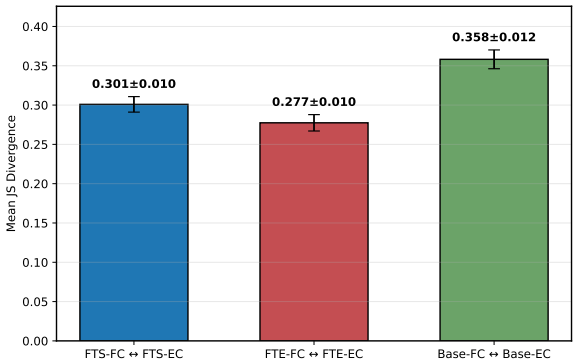

Figure 2: JS divergence of next token logits between FC and EC pairs of FTS and FTE

Hidden State Drift To further probe whether fine-tuning under eviction induces a regime-specific adaptation, we measure how the representation of the final token in each test prompt changes when inference is performed with and without the NAMM policy. For both FTS and FTE, we compute the per-layer ℓ_2 distance between the last-token hidden states obtained under full-cache and evicted-cache decoding and average these distances over the test set. Figure 3 re-

ports, for each layer, the ratio between the drift observed for FTE and that observed for FTS, thereby quantifying the relative sensitivity of the two models' hidden representations to the change in inference regime. Overall, the transition from FTE-FC to FTE-EC exhibits 23.5% greater hidden-state drift than the corresponding transition from FTS-FC to FTS-EC, with the largest differences concentrated in the final three layers. This pattern is consistent with the view that fine-tuning under eviction leads the model to develop representations that are more tightly coupled to the evicted-cache regime. In turn, it supports the interpretation that the gains of FTE-EC over FTS-EC arise from adaptation to that regime rather than from a more general robustness across cache conditions.

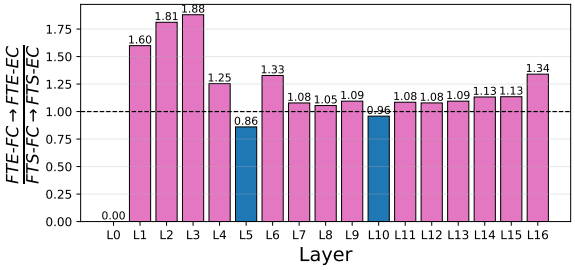

Figure 3: Relative drift of the final token hidden representations under evicted-cache versus full-cache decoding. For each layer, we report the ratio between the ℓ_2 drift observed for FTE (FTE-FC $\rightarrow$ FTE-EC) and that observed for FTS (FTS-FC $\rightarrow$ FTS-EC), averaged over the test set.

Attention Pattern To further characterise the adaptation induced by fine-tuning under eviction, we examine how FTS-FC and FTE-FC distribute attention over the input prompt immediately before answer generation. For each layer, we take the attention weights from the final token to all prompt tokens, map the attended positions back to their location in the original prompt, and compute the fraction of total attention assigned to the first, middle, and last thirds. Figure 4 shows that the two models behave similarly in the first three layers, with little attention allocated to the middle third in either case. From the fourth layer onwards, however, their attention patterns diverge systematically across the first and last thirds of the prompt. On average, FTE-FC places 4.2% more attention mass on the first third of the prompt and 5.0% less on the final third than FTS-FC. This provides complementary evidence that fine-tuning under eviction leads the model to use the available context differently.

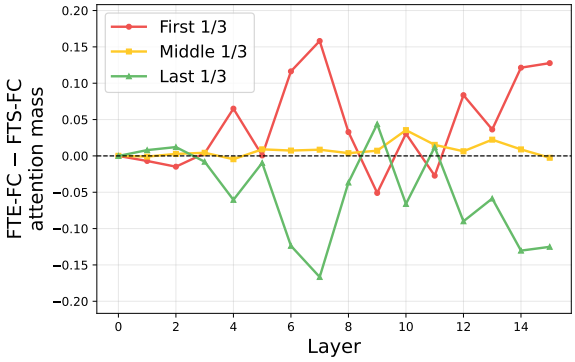

Figure 4: Difference, by layer, in the attention mass that FTE-FC and FTS-FC assign to the first, middle, and last thirds of the prompt under full-cache decoding

5.3 Eviction Mask Stability under Fine-Tuning

We now address the concern introduced in Section 3.3. Because LoRA finetuning inherently modifies a model's internal activations, we expect it to induce a shift in the features used by the eviction policy to prune the cache without the latter being able to adapt to these changes. We then aim to quantify how the LoRA-related drift in the model's internal representations affects the stability of the KV cache pruning policy and how this relationship may change as the LoRA adapters are allowed to actively adapt to the eviction policy. To do so, we evaluate three different model + eviction policy variants, namely Base-EC, FTS-EC and FTE-EC under the same NAMM eviction policy. For each variant, we record the per-layer set of retained tokens and quantify the overlap of these sets across variants using the Jaccard index averaged over the held-out test set, as reported in Figure 5.

Base-EC and FTS-EC share 95.2% of their retained tokens suggesting that the task specific LoRA finetuning does not perturb the model's internal representation in a way which induces significant drift in the eviction mask applied to the KV cache. We note that Base-FC and FTS-FC both suffer the same 43% loss in F1 score when the eviction policy is applied to the models post hoc, as previously seen in Figure 1. In contrast, FTE-EC shares 78.2% of retained tokens with Base-EC and 78.4% with FTS-EC. This corresponds to $\approx$ 22% drift in the eviction mask produced by our eviction policy when LoRA adapts the model's internal representation to the KV cache pruning

regime it operates under.

Effectively, we observe that optimising the standard negative log-likelihood to fine-tune a full cache model on QA tasks does not significantly affect the resulting token eviction dynamics induced by a post-hoc applied eviction policy. On the other hand, our results show that maximising this objective under an active eviction policy induces the representations fed to the latter to drift away from what was seen in Base-EC and FTS-EC. This does not cause any catastrophic instability in fine tuning, as seen by the significant recovery in performance. However, we hypothesise that this drift in representations reduces the performance of the FTE-FC model as it only marginally improves over FTE-EC. As LoRA finetuning progresses, the eviction policy operates on representations which stray away from those it was trained on. This train-test mismatch of the eviction policy could result in some performance degradation, and could explain

against the representations it is fed is thought to force the model representations to drift away from what was seen in FTS, forcing specialisation as previously mentioned. As such, we hypothesise that redistributing the change in parameters induced by finetuning over both LLM and eviction policy networks through joint adaptation, could yield a model more robust to KV-cache pruning regimes without negatively impacting its full cache performance.

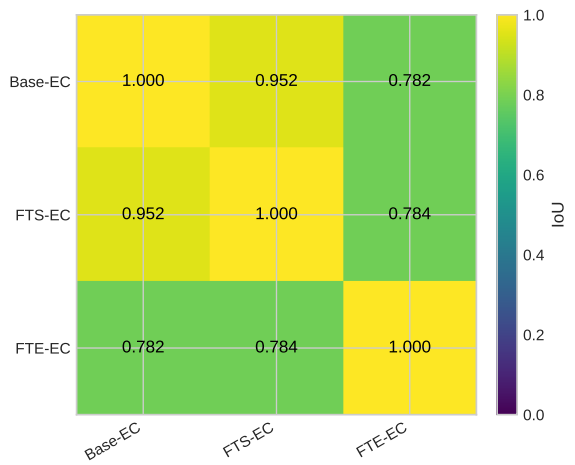

Figure 5: Mean Jaccard index (IoU) Between the Retained Token Sets Generated by Base-EC, FTS-EC & FTE-EC

6 Conclusion

In this report, we investigated the distribution shifts induced by an aggressive KV cache eviction policy and propose eviction-aware adaptation through LoRA finetuning as a method to mitigate the resultant performance degradation of said distribution shifts. We found that performing aggressive KV-cache pruning (75-85% of the tokens) solely by applying the eviction policy post-hoc at inference time led to a collapse in performance on both base and standard fine-tuned models, with the latter degradation reverting any gains introduced by standard fine-tuning. This performance collapse has been shown to be mostly mitigated by eviction-aware fine tuning, with a significantly smaller performance penalty observed under aggressive eviction at inference time. However, we also found that this eviction-aware fine-tuning led to specialisation of the model towards the eviction regime, taking away its ability to take advantage of the full prompt context when the eviction policy is removed. Effectively, this has been empirically inferred from ablations as well as directly shown in the model's internal mechanisms.

Nevertheless, our investigation's face two main limitations, primarily related to their generalisability.

Dataset and Model Limitations Due to compute constraints, our training was limited in model size and in prompt lengths. As explained in 4.2, our dataset is filtered to include examples within a specific token range to allow for sufficiently long contexts for our analysis but are sufficiently short to be manageable within our compute budget. As such, we have not tested the performance of our framework under more aggressive cache compression regimes and cannot infer.

Further, we focused only on QA datasets as a means of easily standardising and evaluating responses. Testing on other tasks such as summarisation and coding, and testing on longer and shorter contexts would enable us to draw more generalised conclusions outside of the specific data used in this study. Similarly, we faced limitations with regards to the model that could be used in this work — all work was conducted on the Llama 3.2 1B model. Further experimentation on larger model sizes and different model families would allow us to draw model-agnostic conclusions.

Eviction Policy Limitations Our analysis in this work is limited to a specific eviction policy, the NAMM, as described in 4.1. Furthermore, we limited our analysis to a specific regime of highly aggressive eviction, with a maximum token budget of 1024 tokens (see Appendix C.1). Thus, our findings cannot be generalised to all eviction policies or all token budgets. Further analysis is required on other 'smart' eviction policies (Zhang et al., 2023; Liu et al., 2023; Zhang et al., 2025), and at different token budgets. Furthermore, the performance of the LLM is dependent on the pretrained NAMM for which we did not investigate training improvements.

Although possibly limited in breadth, our findings indicate that aggressive KV cache compression through eviction can be achieved with minimal performance penalties through eviction-aware fine-tuning, but at the cost of specialisation. Eviction policies such as the NAMM should not be seen solely as inference-time methodologies but must be introduced during training to optimise the memory-performance tradeoff.

Future Work Future works should first attempt to generalise our findings by addressing the limitations in our analysis. Beyond this, for learned eviction policies, our analysis in 5.3 confirms an eviction policy drift during training predicted in 3.3. This suggests further scope for performance improvements by accounting for this drift, which could be achieved by joint fine tuning of the LLM and the eviction policy.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. 2015. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, volume 28.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Edoardo Cetin, Qi Sun, Tianyu Zhao, and Yujin Tang. 2025. An evolved universal transformer memory. In *International Conference on Learning Representations*.

Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, Mohamed S. Abdelfattah, and Kai-Chiang Wu. 2025. Palu: KV-cache compression with low-rank projection. In *International Conference on Learning Representations*.

Yilong Chen, Guoxia Wang, Junyuan Shang, Shiyao Cui, Zhenyu Zhang, Tingwen Liu, Shuohuan Wang, Yu Sun, Dianhai Yu, and Hua Wu. 2024. NACL: A general and effective KV cache eviction framework for LLM at inference time. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7913–7926.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4599–4610.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. 2020. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2790–2799. PMLR.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, volume 37, pages 22947–22970.

Mengqi Liao, Lu Wang, Chaoyun Zhang, Zekai Shen, Xiaowei Mao, Si Qin, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Huaiyu Wan. 2025. G-KV: Decoding-time KV cache eviction with global attention. *arXiv preprint arXiv:2512.00504*.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems*, volume 36, pages 52342–52364.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024b. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 32332–32344. PMLR.

Jose G. Moreno-Torres, Troy Raeder, Rocío Alaiz-Rodríguez, Nitesh V. Chawla, and Francisco Herrera. 2012. A unifying view on dataset shift in classification. *Pattern Recognition*, 45(1):521–530.

Sinno Jialin Pan and Qiang Yang. 2009. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. 2024. Model tells you where to merge: Adaptive KV cache merging for LLMs on long-context tasks. *arXiv preprint arXiv:2407.08454*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, volume 33, pages 17283–17297.

Haoyue Zhang, Hualei Zhang, Xiaosong Ma, Jie Zhang, and Song Guo. 2025. LazyEviction: Lagged KV eviction with attention pattern observation for efficient long reasoning. *arXiv preprint arXiv:2506.15969*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 34661–34710.

A Eviction Mask Stability: Additional Figures

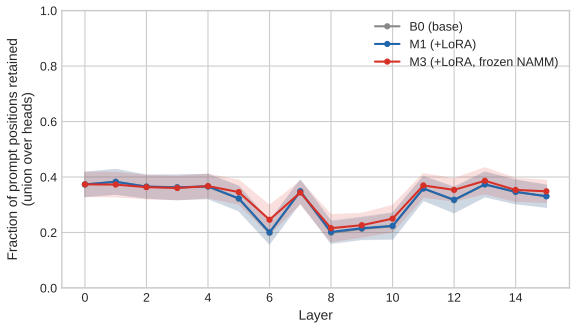

Figure 6: Fraction of prompt positions retained at each layer (union over heads) under the frozen NAMM with Base, FTS, and FTE as the underlying LLM. Retention rates per layer are essentially invariant across the three conditions.

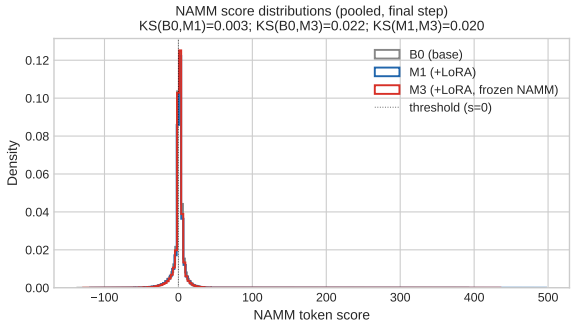

Figure 7: NAMM raw token-score distributions (pooled over heads, layers, and the 70-prompt test split) under Base, FTS, and FTE as the underlying LLM. The dotted vertical line marks NAMM's decision threshold at $s = 0$. The three distributions are nearly identical (pairwise Kolmogorov–Smirnov distances ≤ 0.022).

B Hyperparameters and Training Details

This appendix documents the hyperparameters used to train the NAMM eviction policy (Table 1) and the LoRA adapters for the FTS and FTE model variants (Table 2).

NAMM Eviction Policy Table 1 lists the NAMM training configuration. The NAMM architecture and feature extraction pipeline follow Cetin et al. (2025); here we report the settings used to train the checkpoint used throughout our investigation to perform KV cache pruning for the following models: Base-EC, FTS-EC and FTE-EC. This same trained checkpoint is then used for in post-hoc eviction evaluation and during LoRA finetuning for the FTE model.

LoRA. Table 2 lists the LoRA fine-tuning hyperparameters for FTS and FTE. The two conditions share identical settings except for the presence of the frozen NAMM during training, which requires setting the KV cache size constraint. All other optimisation, data, and architecture settings are matched to isolate the effect of training under eviction. Model checkpoints are selected by validation F1.

CMA-ES	
Population size	8
Samples per batch	8
Training batch size	12
Max iterations	200
Eval interval	5
Scoring initialiser	0
Cache & policy schedule	
Cache size K	1024
Max memory length	1024
Policy firing interval	every 256 tokens
Processing chunk size	256 tokens
Attention spectrogram (STFT)	
FFT size n_{FFT}	32
Hop length	16
Window	Hann, length 32, periodic
Output	magnitude spectrogram
Spectrogram reduction	
Mode	EMA
EMA coefficient	0.99
Learned coefficient	No
BAM scoring network	
Hidden dim	32
Attention heads	1
Bias	Yes
RoPE	No
Masking	Backward
Data	
Conditioning length range	$[4096, 6500]$
Max answer tokens	64
Train / val / test split	70/15/15

Table 1: NAMM training hyperparameters (CMA-ES over the BAM scoring network)

C Implementation Differences from the Original NAMM

C.1 Cache-Size-Based Eviction

Our implementation of NAMM differs from the formulation in Cetin et al. (2025) in how the retention decision is enforced. The original NAMM

	FTS	FTE
LoRA		
Rank	8	8
Alpha	16	16
Dropout	0.05	0.05
Target modules	q,v proj	q,v proj
Optimisation		
Learning rate	1×10^{-4}	1×10^{-4}
Weight decay	0.01	0.01
Max grad norm	1.0	1.0
Warmup ratio	0.03	0.03
Epochs	150	150
Micro-batch size	4	4
Grad. accumulation steps	4	4
Effective batch size	16	16
Early-stopping patience	20	20
Precision	bfloat16	bfloat16
Data		
Max sequence length	7000	7000
Max answer tokens	64	64
Train / val / test split	70/15/15	70/15/15
Split seed	42	42
NAMM		
Active during training	No	Yes
Cache size K	–	1024
Evaluation		
Decoding	greedy	greedy
Temperature	0	0
Max new tokens	64	64

Table 2: LoRA fine-tuning hyperparameters for FTS (fine-tuned standard) and FTE (fine-tuned with eviction)

applies a purely threshold-based rule: at each eviction step, tokens with a learned score $s_i < 0$ are discarded, and all tokens with a score above the threshold $s_i \geq 0$ are retained. Under this rule, the size of the KV cache is not specified in advance and varies across layers, heads, and decoding steps according to the distribution of scores produced by the scoring network.

We instead use a cache-size-based (top-K) rule in which a fixed cache size K, set to 1024 in all our experiments, acts as a hard ceiling on the number of retained tokens per head and layer. Concretely, at each eviction step, tokens with $s_i < 0$ are first pruned as in the original formulation; if more than K tokens remain above the threshold, the surplus is additionally evicted by retaining only the K highest-scoring entries. This guarantees a predictable, upper-bounded memory footprint throughout decoding, which is required both for training the LoRA adapters under a fixed computational budget during FTE fine-tuning (Section 4.1) and for specifying a well-defined compression setting in which to compare model variants (Section 5). In preliminary experiments, we were also unable to reproduce the convergence behaviour of the threshold-only rule reported in Cetin et al. (2025) within our compute budget (population size 8, 200 CMA-ES generations, approximately 1.5 days per run on a single GPU); we attribute this primarily to the reduced search budget relative to the original paper's setup rather than to a methodological difference but cannot rule out additional factors.

C.2 Attention Mask Correction

We identified and fixed a bug in how the attention mask applied over the KV cache tokens was constructed in the original NAMM codebase developed by Sakana AI, which can be accessed here. As mentioned in Section 4.1, the NAMM processes inputs in chunks of 256 tokens, and as new tokens get added to the KV cache, the eviction policy prunes it accordingly. However, the causal attention mask applied to the KV cache tracked the cumulative prompt length rather than the size of the KV cache after eviction. After processing multiple prompt chunks, the per-layer mask slice applied to the attention map was therefore able to grow beyond the size of the effective retained KV cache, to the point where all valid token positions in the cache were eventually masked out. This led to completely uniform attention across layers of the model as the remaining chunks of the prompt were being processed: from approximately the ninth eviction step onwards during chunk processing, every attention head attended uniformly over the surviving cache entries. The NAMM scoring network, which leverages attention spectrogram features as input, was then forced to score tokens based on these uniform attention maps. We fix this by detecting when the width of the attention mask becomes mismatched with the chosen KV cache size and reconstructing a fresh causal mask of the appropriate size, with cache positions unmasked so that attention operates over the correct set of surviving tokens after eviction. We note that generation was not affected by this bug, since the mask is rebuilt from scratch for each newly generated token.